System Programming Project 2

담당 교수 : 김영재 교수님

이름 : 김민우

학번 : 20212019

1. 개발 목표

- 이번 프로젝트에서는 System-level에서의 프로세스 관리, Signaling, Background와 Foreground에서의 실행과 같은 개념을 배우고 익숙해지는 데 의의가 있다.
- Built-in command를 포함한 기본 Shell을 구현하고, Pipe and redirection과 Job의 Background 실행을 추가로 구현하여 Shell 완성도를 높이려 한다.

2. 개발 범위 및 내용

A. 개발 범위

1. Phase 1

   기본적인 내부 shell command 실행이 되어야 한다. 해당 command로는 cd, ls, mkdir, rmdir, touch, cat, echo, exit이 있다.

2. Phase 2

   Dup2를 통해 Pipe command 실행이 가능하도록 한다. 다음 Pipe 명령어에서 이전 출력 결과를 받아 처리가 가능해야 한다.

3. Phase 3

   Process를 Background에서 실행할 수 있도록 해야 한다. 이때 Process state를 바꾸면서 Job을 Background와 Foreground를 오갈 수 있도록 한다.

B. 개발 내용

- **Phase1 (fork & signal)**

  ✓ Fork() 를 통해 Child process를 생성하고 Child process에서 Job이 수행될 수 있도록 한다. Execvp 함수를 통해 Job을 수행하고 정상적으로 Signal 출력되었으면 return value가 0~255로 나올 것이고, 결과가 부합하지 않으면 에러 메시지를 출력하고 종료하게 한다.

  ✓ Child process에서 연결 종료 시에 SIGCHLD를 발생시켜 이를 Parent process에 전달하게 되는데, 이때 signal 함수를 통해 미리 SIGCHLD signal과 연결해 놓은 sigchld_handler에서 Signal을 Handling하게 된다. 정상적으로 종료된 경우에는 Job struct를 초기화시키고 STOPPED이거나 CONTINUED인 경우에도

경우에 따라 적절히 처리해주면 된다. Parent process에서는 PID를 addjob으로 저장해두고 sigsuspend를 통해 SIGCHLD를 기다리게 된다. SIGCHLD 발생 시 eval 함수를 return 할 수 있게 된다.

- **Phase2 (pipelining)**
  - ✓ Command line으로부터 '|' 문자를 찾고 이를 가지고 명령어를 분할하여 개수는 cmdcmt에, 명령어는 cmdpipe에 저장한다.
  - ✓ Pipeline 개수를 미리 cmdcnt에 저장하였으므로, 이 개수를 가지고 main에서 순차적으로 cmdpipe에 저장된 command를 실행하도록 한다. 이때 명령 사이의 표준 입력과 표준 출력을 그대로 연결해주어야 하므로 Dup2를 이용해 준다.
- **Phase3 (background process)**
  - ✓ Background process를 나타내는 Ampersand를 Command line parsing 하는 함수 parse_line에서 찾는다. 명령어 맨 마지막에서 ampersand를 찾을 경우 Bool 형태의 flag를 표시해두었다가, 표시된 명령어를 시행할 때에는 Parent process에서 해당 mask를 기다리지 않고 넘어가며 pid와 command를 순서대로 출력하여 Background임을 나타낸다.

C. 개발 방법

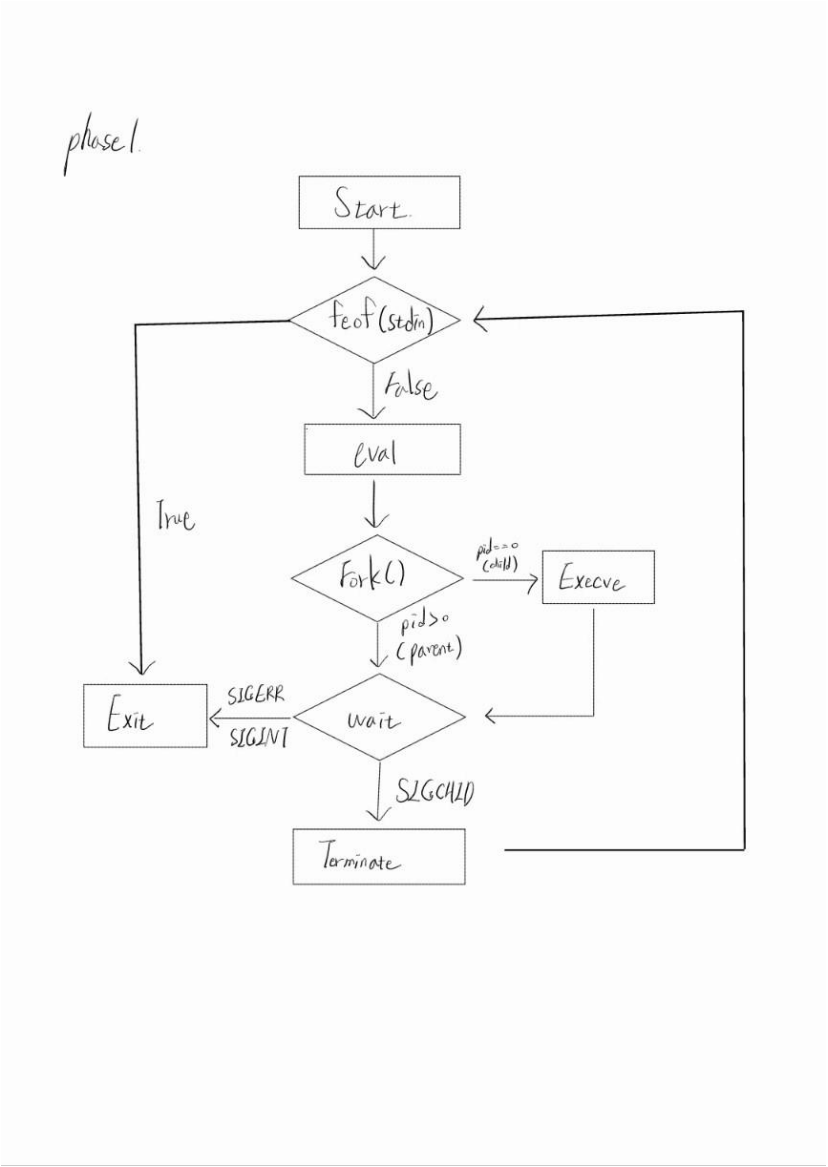
- 우선 Job struct를 만들어 Process 관리가 용이하도록 하였다. Job struct는 pid와 state, background flag, command를 포함하도록 하였다. Command line을 받기 위해 string type의 cmdline과 앞에서 언급했던 cmdpipe를 선언해주고, cmdcnt를 전역 변수 선언하여 pipe 개수를 알 수 있도록 하였다. 이후 main 함수의 구조를 구축하였다. Job 배열을 초기화한 후 while 문을 통해 shell에서 명령을 계속 수행할 수 있도록 한다. Shell prompt 출력 후 stdin에서 fgets를 통해 명령어를 읽어주고 이 명령어를 parse_pipe에서 처리하여 독립된 명령어 배열로 분리한다. 분리된 배열은 for 문을 통해 eval에서 수행되게 되는데, 우선 parse_line을 통해 명령어를 처리해준다. 양쪽 trim을 수행하고 whilespace로 분리하여 명령을 수행할 준비를 하고, child process에서 execvp를 통해 명령을 수행한다. Parent process에서 SIGCHLD를 처리하기 위해 sigchld_handler가 필요하다. Sigchld_handler에서는 기다렸던 job을 초기화하거나 state를 바꿔준다. 이때 job 관리에 필요한 각종 함수

가 필요하게 된다. 이 함수들은 update, delete, addjob, show_jobs, kill_job, find_jobnum 등으로 구성되어 있다. 이 함수들은 Job struct 배열에서 job들을 관리하도록 구현되었다.

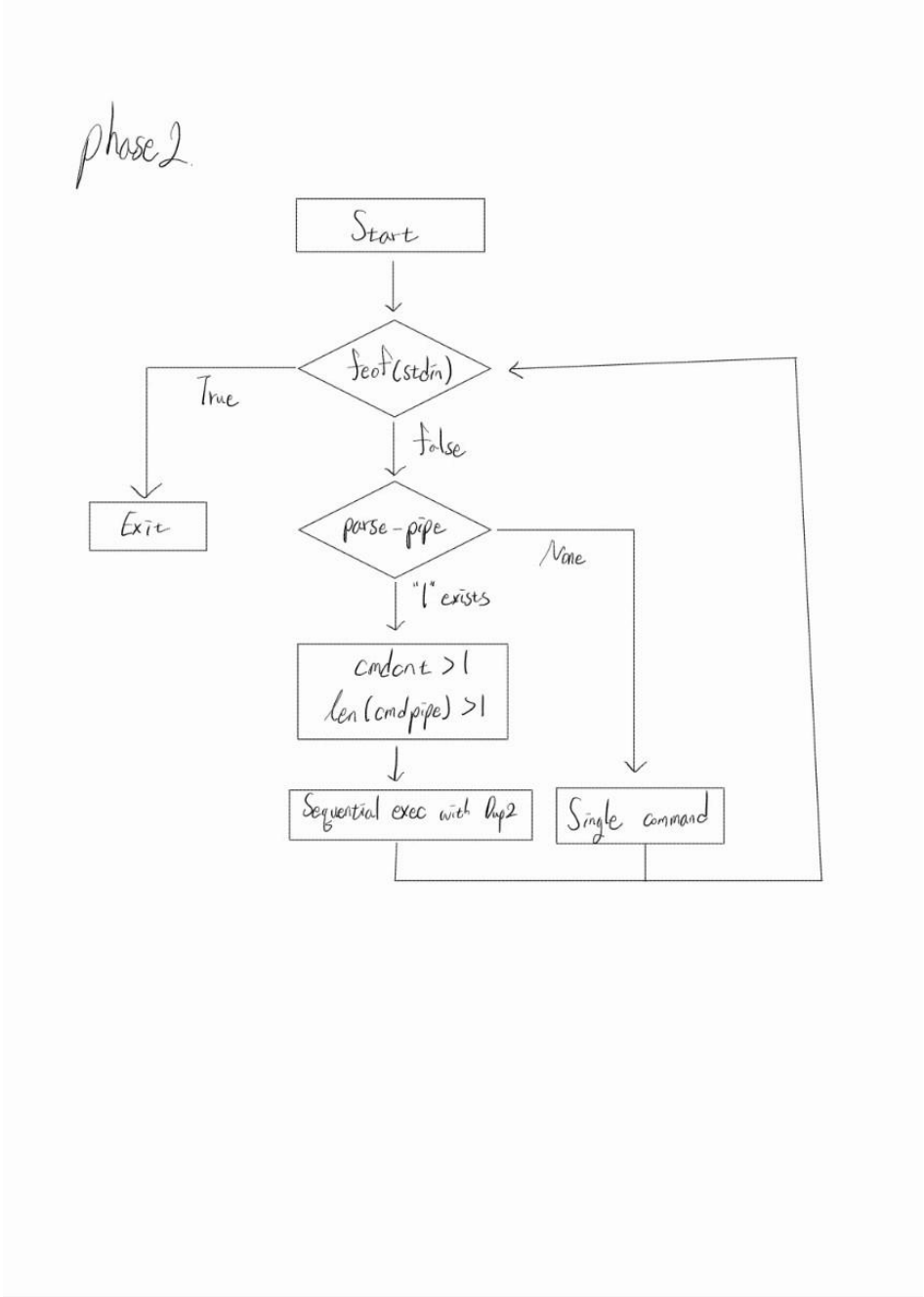
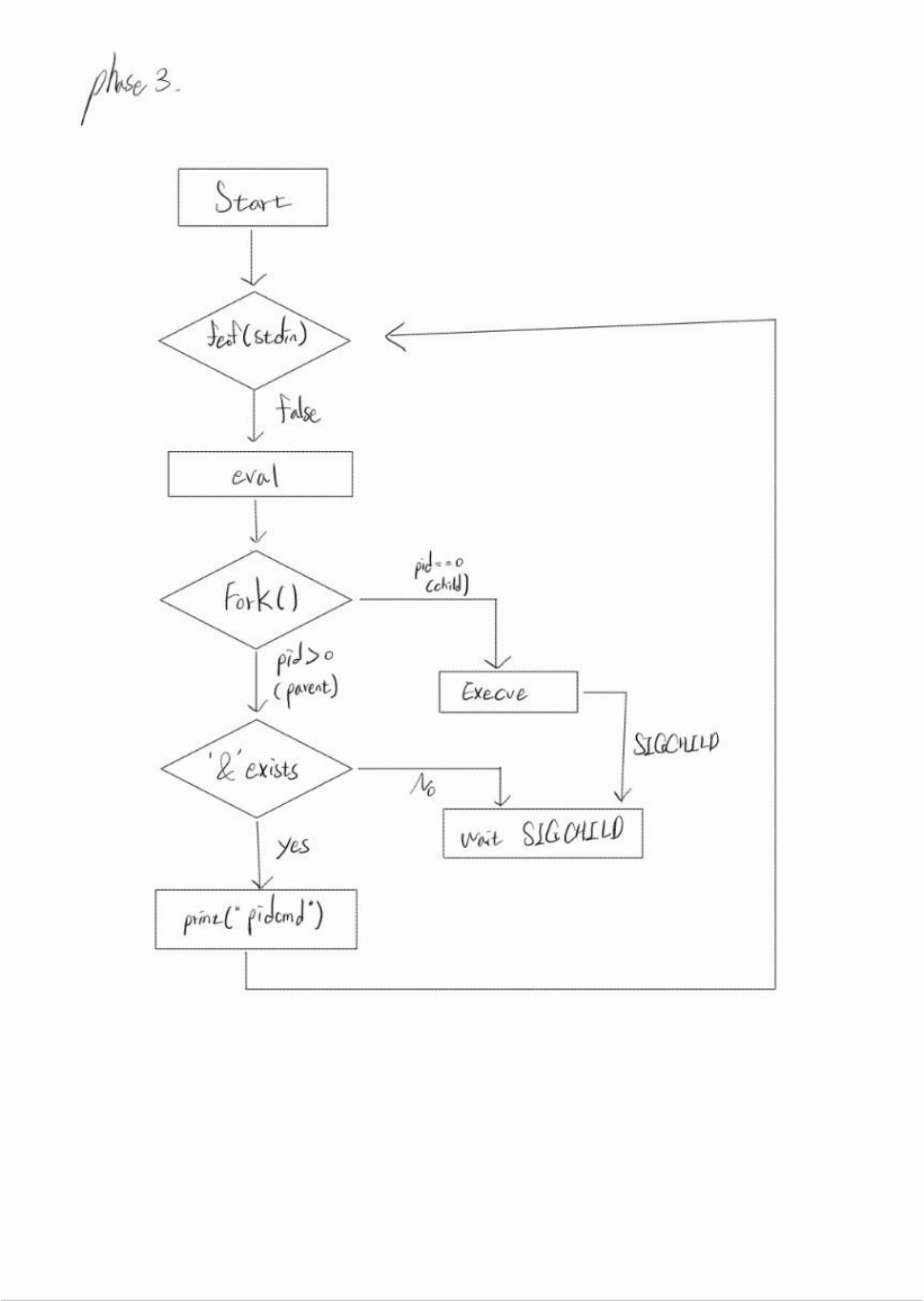
3. 구현 결과

A. Flow Chart

1. Phase 1 (fork)



2. Phase 2 (pipeline)



3. Phase 3 (background)